

Measuring Student Over-Reliance on AI in Programming Labs: A Simple Behavioral Index (BORI) in E-STRANGE

Research Proposal: Core Concept, Automated Telemetry Pipeline, and Statistical Validation

SUMMARY IN BRIEF

This study measures how beginner students use AI assistants in lab assignments: do they ask for explanations to learn (constructive), or do they blindly copy-paste full code (over-reliance)? We calculate an automated **Behavioral Over-Reliance Index (BORI)** from existing server logs without disturbing students. We then test whether over-reliance harms closed-book exam scores and whether a 1-sentence reflection prompt helps students think more independently.

1. Why This Study Matters

Generative AI tools help students solve programming tasks faster, but how students ask for help matters greatly [4], [9]:

- **Healthy Help-Seeking (Instrumental):** Asking AI for conceptual hints, debugging guidance, or logic explanations. The student remains in charge of writing the code [1], [7].
- **Over-Reliance (Executive):** Asking AI for complete solutions and copying them directly into the editor without thinking [1], [2].

Prior studies suggest students who lean heavily on blind copy-pasting may struggle when taking unassisted, closed-book exams [2], [3]. Standard chat interfaces lack reflection friction, which can encourage fast, unread pasting [1], [10].

2. The BORI Formula (Simple & Intuitive)

BORI is a single number from **0.0 (fully independent / healthy)** to **1.0 (total uncritical over-reliance)**. It is simply the equal-weighted average of 4 observable behaviors [6], [8]:

$$\text{BORI}_{\text{prompt}} = \frac{E_{\text{direct}} + S_{\text{copy}} + P_{\text{paste}} + F_{\text{fast}}}{4}$$

Each component is scored from 0.0 to 1.0 • If AI returns text/hints only (no code), BORI is the average of E_{direct} and S_{copy} (50% each) • Zero AI use is logged as missing.

1. Direct Code Request ($E_{\text{direct}} \in \{0, 1\}$)

Did the student ask for complete solution code (1) or ask for a concept/hint (0)? Classified locally via a Llama-3.1-8B model with regex rules (validated on 200 pilot prompts, $\kappa_{\text{human}} \geq 0.85$).

2. Problem Text Copying ($S_{\text{copy}} \in [0, 1]$)

How much of the prompt is verbatim copied from the problem description? Measured by 4-gram overlap (0 = student phrased own words, 1 = exact copy-paste of assignment).

3. Unedited Code Pasting ($P_{\text{paste}} \in [0, 1]$)

Did the student paste the AI's code snippet directly into their editor? Measured via normalized edit distance (1 = exact paste without any edits, 0 = heavily modified).

4. Fast Paste Without Reading ($F_{\text{fast}} \in [0, 1]$)

Did the student paste within seconds without reading?
 $F_{\text{fast}} = 1 - \min(1.0, T_{\text{active}} / T_{\text{expected}})$, where active time subtracts tab-blur/defocus time and expected reading speed is 5 tokens/s.

3. How Data is Collected (Zero Extra Work for Students)

All data is pulled automatically from existing E-STRANGE server logs and course records:

Data Source	What is Recorded	Purpose in Study
1. Chat Logs	Prompt text, AI response, response token count, timestamp.	Calculates Direct Code Request (E_{direct}) and Problem Copying (S_{copy}).
2. Editor & Tab Logs	Submitted code, paste timestamp, tab-defocus (blur) duration.	Calculates Unedited Pasting (P_{paste}) and Reading Time (F_{fast}).
3. Course Grades & Surveys	Pre-test ability (Z_c), Hou et al. [5] survey (W3/W6), and exam score (Y_{exam}).	Validates BORI against survey and evaluates unassisted exam impact.

4. Research Questions & Statistical Testing Plan

We answer 3 practical questions using standard statistical tests. **RQ3 is our primary confirmatory test**, while RQ1 and RQ2 are exploratory pilot analyses:

Research Question	Statistical Test & Model	What We Expect to Learn
RQ1: Exam Impact Does high over-reliance predict lower exam scores?	Hierarchical Regression: $Y_{\text{exam}} = \beta_0 + \beta_1 Z_{\text{pretest}} + \beta_2 \text{BORI}$	Checks if BORI explains exam performance after controlling for prior student ability ($\beta_2 < 0$).
RQ2: Reflection Gate Does a 1-sentence prompt reduce over-reliance?	Linear Mixed Model (LMM): Compares BORI before vs after reflection gate.	Tests if requiring students to write a bug hypothesis before seeing AI code reduces BORI ($\beta_{\text{phase}} < 0$).
RQ3: Survey Alignment (Primary Confirmatory Test)	Pearson / Spearman Correlation: BORI vs Hou et al. (2025) [5] Reliance Scale.	Confirms that server-measured BORI matches students' self-reported reliance ($r > 0, p < 0.05$).

5. Study Setup, Sample Size & Privacy

- **Setting & Sample:** 1 introductory lab section ($N \approx 45$ enrolled; ~ 38 active after accounting for absence and zero-use). For a moderate-to-large correlation ($r = 0.40$), statistical power is 70.8% at $N =$ (reaching 80% if expanded to 2 sections, $N \geq 70$).
- **Reflection Gate Intervention:** In Modules 3–4, before AI responses appear, students must write 1 brief sentence stating their bug hypothesis or coding plan. Low-effort text (e.g. repeated keystrokes) is tracked.
- **Privacy & Ethics:** Prompt classification runs 100% locally on the lab server (no cloud API). Student survey participation is managed by an independent research assistant and masked from the instructor until final grades are submitted.

6. Lab Schedule & Collaboration

Lab Schedule (6 Weeks) Week 1–2 (Baseline): Modules 1–2 with standard AI + Mid-Survey. Week 3–4 (Intervention): Modules 3–4 with Reflection Gate + Post-Survey. Week 5: Unassisted closed-book lab exam. Week 6: Automated data extraction and statistical testing.	Faculty Collaboration (FTRC) 1. Deploy study in 1 lab section (2 sections if feasible). 2. No changes required to existing teaching materials. 3. Joint co-authorship on resulting research papers.
--	---

References

[1] M. Liffiton, B. E. Sheese, J. Savelka, and P. Denny, "CodeHelp: Using large language models with guardrails for scalable support in programming classes," in *Proc. 23rd Koli Calling Int. Conf. Comput. Educ. Res. (Koli Calling '23)*, 2023, pp. 1–11, doi: 10.1145/3631802.3631830.

[2] J. Prather, B. N. Reeves, P. Denny, B. A. Becker, J. Leinonen, A. Luxton-Reilly, G. Powell, J. Finnie-Ansley, and E. A. Santos, "'It's weird that it knows what I want': Usability and interactions with Copilot for novice programmers," *ACM Trans. Comput.-Hum. Interact.*, vol. 31, no. 1, pp. 1–31, 2024, doi: 10.1145/3617367.

[3] H. Bastani, O. Bastani, A. Sungu, H. Ge, Ö. Kabakcı, and R. Mariman, "Generative AI without guardrails can harm learning: Evidence from high school mathematics," *Proc. Natl. Acad. Sci. U.S.A.*, vol. 122, no. 26, Art. no. e2422633122, 2025, doi: 10.1073/pnas.2422633122.

[4] M. Kazemitabaar, J. Chow, C. K. T. Ma, B. J. Ericson, D. Weintrop, and T. Grossman, "Studying the effect of AI code generators on supporting novice learners in introductory programming," in *Proc. 2023 CHI Conf. Hum. Factors Comput. Syst. (CHI '23)*, 2023, pp. 1–23, doi: 10.1145/3544548.3580919.

[5] C. Hou, G. Zhu, V. Sudarshan, F. S. Lim, and Y. S. Ong, "Measuring undergraduate students' reliance on generative AI during problem-solving: Scale development and validation," *Comput. Educ.*, vol. 234, Art. no. 105329, 2025, doi: 10.1016/j.compedu.2025.105329.

[6] R. M. Dawes, "The robust beauty of improper linear models in decision making," *Amer. Psychol.*, vol. 34, no. 7, pp. 571–582, 1979, doi: 10.1037/0003-066X.34.7.571.

[7] S. Nelson-LeGall, "Help-seeking: An active instrumental strategy for learning," *J. Educ. Psychol.*, vol. 73, no. 6, pp. 813–824, 1981, doi: 10.1037/0022-0663.73.6.813.

[8] A. Diamantopoulos and H. M. Winklhofer, "Index construction with formative indicators: An alternative to scale development," *J. Marketing Res.*, vol. 38, no. 2, pp. 269–277, 2001, doi: 10.1509/jmkr.38.2.269.18845.

[9] T. W. Price, Z. Liu, V. Cateté, and T. Barnes, "Factors influencing students' help-seeking behavior while programming with human and computer tutors," in *Proc. 2017 ACM Conf. Int. Comput. Educ. Res. (ICER '17)*, 2017, pp. 127–135, doi: 10.1145/3105726.3106179.

[10] P. Denny, J. Leinonen, J. Prather, A. Luxton-Reilly, T. Amarouche, B. A. Becker, and B. N. Reeves, "Prompt Problems: A new programming exercise for the generative AI era," in *Proc. 55th ACM Tech. Symp. Comput. Sci. Educ. (SIGCSE 2024)*, 2024, vol. 1, pp. 296–302, doi: 10.1145/3626252.3630909.